There are multiple choices related to distributed tracing systems. First, we must choose the format and protocol we'll use to generate and propagate traces. For this we'll use OpenTelemetry (also called *OTel*), a CNCF-incubating project that is quickly becoming the de facto standard for distributed tracing and aims at unifying the collection of telemetry data (https://opentelemetry.io).

Next we need to choose whether to use OpenTelemetry directly (with the OpenTelemetry Java instrumentation) or rely on a façade that instruments the code in a vendor-neutral way and integrates with different distributed tracing systems (such as Spring Cloud Sleuth). We'll go with the first option.

Once the applications are instrumented for distributed tracing, we'll need a tool to collect and store traces. In the Grafana observability stack, the distributed tracing backend of choice is Tempo, a project that "lets you scale tracing as far as possible with minimal operational cost and less complexity than ever before" (https://grafana.com/oss/tempo). Unlike the way we used Prometheus, Tempo follows a push-based strategy where the application itself pushes data to the distributed tracing backend.

This section will show you how to complete the Grafana observability setup with Tempo and use it to collect and store traces. Then I'll show you how to use the OpenTelemetry Java instrumentation in your Spring Boot applications to generate and send traces to Tempo. Finally, you'll learn how to query traces from Grafana.

OpenTelemetry, Spring Cloud Sleuth, and Micrometer Tracing